

SPRÁVCE HESEL S DVOUFAKTORVOU AUTENTIZACÍ

Kožuškanič Oliver (278995)
Chalány Andrej (259832)
Foralová Aneta (277788)
Bugová Alžbeta (259498)

OBSAH

Popis aplikace	3
Účel aplikace	3
Hlavní cíle	3
Co aplikace umožňuje?	3
Popis vstupu	4
Vstupní soubory.....	4
Výstupní soubory.....	4
Potřebné soubory pro spuštění a fungování aplikace	5
Seznam použití externích knihoven	5
Popis implementovaných algoritmů	6
Hašování přístupového hesla uživatele – PBKDF2-SHA256	6
Šifrování hesel ke službám – Fernet (AES-128-CBC)	6
Dvoufaktorová autentizace – TOTP (RFC 6238)	6
Kontrola integrity hesel – HMAC-SHA256	6
Kontrola odolnosti hesla	7
Šifrovaný auditní log	7
Vývojový diagram	8
Architektura řešení.....	9
Přehled architektury	9
Komponenty a jejich role.....	9
Vrstva Prezentace (GUI)	9
Vrstva Aplikační Logiky.....	9
Vrstva Kryptografie.....	10
Vrstva databází	11
Tok dat v aplikaci	11
Registrace nového uživatele	11
Přihlášení uživatele.....	12
Uložení hesla ke službě	12
Čtení (dešifrování) hesla	13
Nastavení 2FA	13
Bezpečnostní architektura	14
Vrstvy bezpečnosti.....	14
Ochrana hlavního hesla	15

Ochrana hesel ke službám	15
Architektura systému	15
<i>Testovací metodika a vyhodnocení výsledků.....</i>	16
Testovací metodika	16
Vyhodnocení	16

Popis aplikace

Aplikace slouží jako správce hesel s důrazem na bezpečnost, využívající moderní kryptografické metody a dvoufaktorovou autentizaci pro ochranu uživatelských dat.

Aplikace běží na operačních systémech s podporou Pythonu (Windows, macOS, Linux) a poskytuje uživatelsky přívětivé grafické rozhraní pro správu hesel.

Aplikace je určena jednotlivcům, kteří potřebují bezpečně uchovávat hesla ke svým účtům, přičemž si přejí ochranu před ztrátou nebo krádeží dat.

Účel aplikace

Hlavní cíle

1. Bezpečné uchovávání hesel - Všechna hesla jsou šifrována pomocí moderní kryptografie (Fernet - AES-128-CBC) a nemůžou být přečtena bez správného hesla uživatele.
2. Autentizace s 2FA - Aplikace vyžaduje nejen heslo, ale také dvoufaktorovou autentizaci (TOTP) pro přihlášení, což znemožňuje neoprávněný přístup i v případě úniku hesla.
3. Kontrola integrity dat - Každé uložené heslo má přiřazen kontrolní součet (HMAC-SHA256), který zjistí jakoukoliv neoprávněnou změnu dat v databázi.
4. Audit a záznam operací - Aplikace zaznamenává všechny důležité operace (přihlášení, uložení hesla, změny 2FA), což umožňuje ověřit, kdy byla která operace provedena.

Co aplikace umožňuje?

Pro koncového uživatele

- Registrace a přihlášení – Vytvoření nového účtu s silným heslem a nastavení 2FA
- Uložení hesel – Bezpečné uložení hesel ke službám (např. Gmail, Facebook, online bank)
- Správa hesel – Úprava, smazání nebo zobrazení uložených hesel
- Dvoufaktorová autentizace – Nastavení TOTP pro zvýšenou bezpečnost
- Záložní kódy- 10 záložních kódů pro případ ztráty 2FA zařízení
- Audit trail – Přehled všech operací provedených v aplikaci

Popis vstupu

Vstupní soubory

Během běhu aplikace uživatel poskytuje následující data:

VSTUP	TYP	PŘÍKLAD	POUŽITÍ
Uživatelské jméno	Text (plaintext)	honza@honza.cz	Identifikace uživatele
Heslo uživatele	Text (tajné)	SecureTry?125/	Autentizace, šifrování
Název služby	Text (plaintext)	vut.cz	Identifikace hesla
Heslo ke službě	Text (tajné)	2457?Vutcz	-
TOTP kód	Číslo	259 854	2FA autentizace
Záložní kódy	Číslo	15896541	V případě ztráty 2FA zařízení na autentizaci

Výstupní soubory

VÝSTUP	KDE	FORMÁT	OBSAH
Grafické rozhraní	Obrazovka zařízení	GUI	Přihlášení a uživatelská správa aplikace
QR kód	Obrazovka zařízení	PNG	Inicializace pro TOTP
Záložní kódy	Obrazovka zařízení	Text	Hesla pro záchranný přístup
Audit log	Obrazovka zařízení	Text	Záznam operací, jednotlivých uživatelů

Aplikace vytváří následující soubory a adresáře:

└─ db¹/ Adresář s databázemi
| └─ users.db Účty uživatelů

¹ Adresář se automaticky vytvoří s první spuštěním aplikace

	— passwords.db	Uložená hesla (šifrovaná)
	— twofa.db	TOTP sekrety a záložní kódy
	— audit.db	Audit log (šifrovaný)

Potřebné soubory pro spuštění a fungování aplikace

main.py	Spuštění aplikace
gui_pyside6.py	Grafické rozhraní
crypto.py	Kryptografické operace
database.py	Správa SQLite databází
user.py	Registrace a přihlášení
password_manager.py	Operace nad hesly
two_factor_auth.py	Správa 2FA
audit.py	Správa a šifrování operací
integrity.py	Kontrola HMAC značek
congif.py	Konfigurace

Seznam použití externích knihoven

Pyside6	Grafické zobrazení (GUI)
Qr code	generování QR kódu pro 2FA
Pillow	Zpracování QR kódů pro 2FA
cryptography	Implementace šifrování a kryptografie
Knihovny obsahované v python verzi	Fungování aplikace

Popis implementovaných algoritmů

Hash přístupového hesla uživatele – PBKDF2-SHA256

Pro uložení uživatelského hesla se používá funkce PBKDF2 (Password-Based Key Derivation Function 2). Před hash je vygenerována náhodná 32bajtová sůl pomocí funkce `secrets.token_hex()`, která zabraňuje útokům typu rainbow table. Heslo je následně hašováno s 100 000 iteracemi, což zpomaluje útoky hrubou silou. Výsledný hash spolu se solí je uložen do databáze `users.db`. Pro ověřování hesla je použita funkce `hmac.compare_digest()` místo běžného porovnání `==`, čímž je zajištěna ochrana proti timing útokům.

Šifrování hesel ke službám – Fernet (AES-128-CBC)

Hesla ke službám musí být uložena reverzibilně, tedy tak, aby je bylo možné dešifrovat. K tomuto účelu je použito šifrování Fernet z knihovny *cryptography*, které interně využívá algoritmus AES-128 v režimu CBC s PKCS7 paddingem a HMAC-SHA256 pro autentizaci zprávy. Stejný mechanismus je použit také pro šifrování záznamů v auditním logu.

Dvoufaktorová autentizace – TOTP (RFC 6238)

Time-based One-Time Password (TOTP) je implementován bez použití externí knihovny. Algoritmus generuje náhodný 20bajtový secret, který je zakódován ve formátu Base32. Tento secret si uživatel naskenuje pomocí QR kódu do autentizační aplikace (např. Google Authenticator).

Při každém přihlášení aplikace vypočítá aktuální časový interval (čas / 30 sekund) a pomocí HMAC-SHA1 vygeneruje 6místný kód, který je porovnán s kódem zadaným uživatelem. Pro toleranci časového posunu jsou kontrolovány i sousední intervaly (`window = 1`). Pro případ ztráty zařízení jsou generovány také jednorázové záložní kódy (celkem 10).

Kontrola integrity hesel – HMAC-SHA256

Při každém uložení hesla ke službě je vygenerován HMAC-SHA256 tag z kombinace šifrovaného hesla, soli a identifikátoru uživatele (`user_id`). Klíčem pro výpočet HMAC je

hodnota AUDIT_ENCRYPTION_KEY kombinovaná s user_id. Tento tag je uložen spolu s heslem v databázi passwords.db.

Při každém načtení hesla je tag znovu vypočítán a porovnán s uloženou hodnotou. Pokud se hodnoty neshodují, je heslo označeno jako poškozené a není vráceno uživateli. Tento mechanismus umožňuje detekovat jakoukoli neoprávněnou změnu dat v databázi.

Kontrola odolnosti hesla

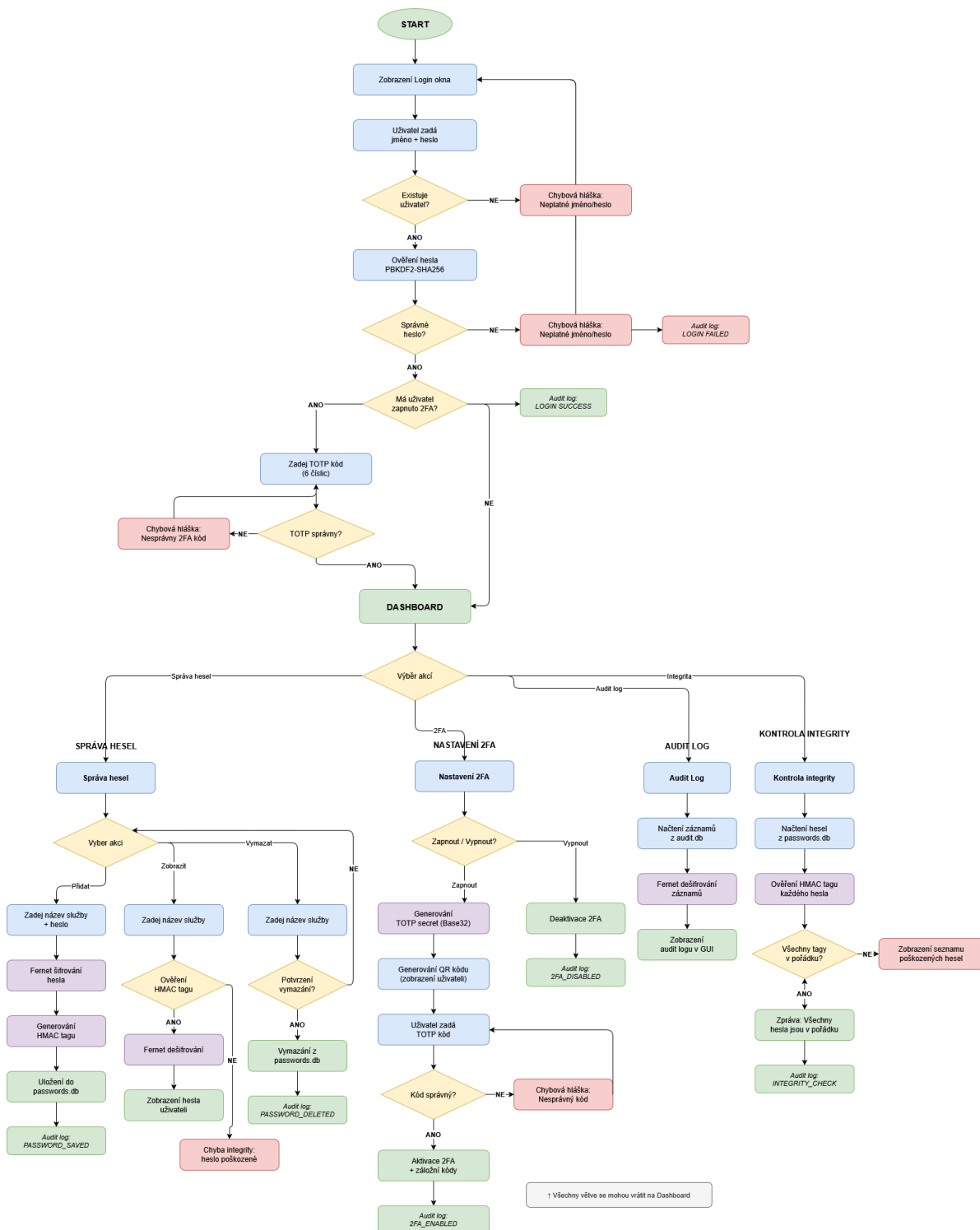
Před registrací i změnou hesla je ověřována jeho síla. Heslo musí mít minimálně 12 znaků a obsahovat alespoň jedno velké písmeno, jedno malé písmeno, číslici a speciální znak. Pokud heslo tyto podmínky nesplňuje, je akce zamítnuta a důvod je zaznamenán do auditního logu.

Šifrovaný auditní log

Každá akce v aplikaci (např. přihlášení, uložení hesla, ověření 2FA nebo chyby) je zaznamenána do databáze audit.db. Samotná akce, její detaily i IP adresa jsou před uložením zašifrovány pomocí mechanismu Fernet, takže při přímém přístupu k databázovému souboru jsou data nečitelná. Dešifrování probíhá až při zobrazení záznamů v grafickém uživatelském rozhraní.

Vývojový diagram

Vývojový diagram — Správce hesel



Architektura řešení

Přehled architektury

Aplikace je postavena na vrstvené architektuře se čtyřmi hlavními vrstvami:

VRSTVA PREZENTACE (GUI – PySide6)
VRSTVA APLIAKČNÍ LOGIKY (správa hesel, 2FA, audit)
VRSTVA KRYPTOGRAFIE (hash, šifrování, TOTP, HMAC)
VRSTVA DAT (SQLite databáze)

Komponenty a jejich role

Tok aplikace: Uživatel → GUI → Aplikační logika → Kryptografie → Databáze

Vrstva Presentace (GUI)

- Soubor: gui_pyside6.py - Odpovídá za interakci s uživatelem prostřednictvím grafického rozhraní.

Komponenty GUI

LoginWindow	Přihlášení/registrace	Vstupní okno aplikace
DashboardWindow	Hlavní okno	Záložky (hesla, 2FA, audit)
2FA verify	Ověření 2FA	TOTP kód + záložní kódy
2FA setup	Nastavení 2FA	QR kód + záložní kódy
BackupCodes	Zobrazení kódů	Vkládání/kopírování záložních hesel

Vrstva Aplikační Logiky

Řídí fungování a tok aplikace bez kryptografických operací.

Třídy:

uživatel	User.py	Registrace, přihlášení, ověření
hesla	Password_manager.py	Operace na heslech
2FA	Two_factor_auth.py	Nastavení a ověřování 2FA
audit	Audit.py	Záznam operací
Intgrita	Integrity.py	Kontrola HMAC značek

Příklad toku:

uživatel → LoginWindow.login() → UserManager.login() → CryptoManager.verify_password()
→ db.get_user() → TwoFAVerifyDialog → success/fail

Vrstva Kryptografie

- Soubor: crypto.py
- Implementuje všechny bezpečnostní algoritmy bez znalosti aplikační logiky.

Kryptografické operace:

PBKDF2-SHA256	Hash_password	Hash hesla uživatele	Verify_passwords	Ověření hesla
Fernet (AES-128)	Encrypt_password	Šifrování hesla k službě	Decrypt_password	Dešifrování hesla
HMAC-SHA256	Generate_integrity_tag	Vytvoření HMAC značky	Verify_integrity_tag	Ověření HMAC značky
HMAC-SHA1+TOTP	Generate_secret_key	Vytvoření TOTP	Verify_totp	Ověření kódu TOTP

- Bezpečnostní vlastnosti:
 - Timing-safe porovnávání (hmac.compare_digest)
 - Náhodné soli pro PBKDF2
 - Bezpečné generování náhodných čísel (secret modul)

Vrstva databází

- Soubor: database.py
- Správa čtyř oddělených SQLite databází.

User.db	Správa uživatelských účtů (PBKDF2)
Passwords.db	Uložená hesla (AES – 128) + HMAC
Twofa.fb	TOTP a záložní kódy
Audit.db	Log operací (AES-128)

QLite databáze v navrženém systému slouží pouze jako úložiště šifrovaných dat a souvisejících metadat. Citlivé informace, jako jsou hesla ke službám, TOTP tajemství nebo auditní záznamy, nejsou ukládány v otevřené podobě.

Pro ochranu dat je využíván symetrický šifrovací mechanismus Fernet z knihovny *cryptography*, který interně využívá algoritmus AES-128 v režimu CBC s PKCS7 paddingem a zároveň implementuje autentizaci zprávy pomocí HMAC-SHA256. Tento přístup zajišťuje jak důvěrnost, tak integritu uložených dat.

Šifrovací klíče jsou spravovány na úrovni aplikace a jsou odvozeny nebo načítány z bezpečných zdrojů (např. konfigurační soubor nebo environmentální proměnné). Bez znalosti těchto klíčů není možné uložená data dešifrovat.

Kromě vestavěné autentizace ve schématu Fernet je nad databázovými záznamy implementována dodatečná kontrola integrity pomocí HMAC-SHA256. Tento mechanismus slouží k ověření, že data nebyla neoprávněně změněna přímo na úrovni databáze. Při každém načtení dat dochází k opětovnému výpočtu HMAC a jeho porovnání s uloženou hodnotou.

Databáze SQLite tedy neobsahuje žádná citlivá data v otevřené podobě. Slouží pouze jako strukturované úložiště, zatímco veškeré bezpečnostní vlastnosti, včetně šifrování, autentizace a kontroly integrity, jsou realizovány na úrovni aplikace.

Tok dat v aplikaci

Registrace nového uživatele

1. Uživatel zadá jméno + heslo → GUI
 2. LoginWindow.register() volá UserManager.register()
 3. UserManager volá CryptoManager.hash_password()
- Generuje náhodný salt

- Hash heslo PBKDF2 (100 000 iterací)
4. Uloží uživatele do users.db
 5. AuditManager zaznamenává akci

Přihlášení uživatele

1. Uživatel zadá heslo → LoginWindow
2. LoginWindow.login() volá UserManager.login()
3. UserManager načte z DB a ověří heslo
 - CryptoManager.verify_password() - timing-safe
4. Pokud OK → TwoFAVerifyDialog
 - Uživatel zadá 6místný TOTP kód
5. TwoFactorAuthManager.verify_totp() ověří kód (RFC 6238)
 - Kontrola času ± 30 sekund
 - Kontrola replay protection
6. Pokud OK → DashboardWindow (úspěšné přihlášení)
7. AuditManager zaznamenává přihlášení

Uložení hesla ke službě

1. Uživatel zadá heslo ke službě → GUI
2. PasswordManager.save_password() je volán
3. CryptoManager.encrypt_password() šifruje heslo
 - Vygeneruje nový Fernet klíč z hesla uživatele
 - Zašifruje heslo služby
4. CryptoManager.generate_integrity_tag() vytvoří HMAC
 - HMAC-SHA256(klíč + user_id, šifrovaná_hesla)
5. Uloží do passwords.db:

- password_hash (Fernet šifrované)
- salt (pro Fernet klíč)
- integrity_tag (HMAC-SHA256)

6. AuditManager zaznamenává uložení

7. IntegrityManager.verify_password_integrity() ověří HMAC

Čtení (dešifrování) hesla

1. Uživatel klikne na heslo → GUI
2. PasswordManager.get_password() je volán
3. IntegrityManager.verify_password_integrity() ověří HMAC
 - Porovná uložený HMAC s vypočítaným
 - Pokud se liší → INTEGRITY FAILED
4. Pokud OK → CryptoManager.decrypt_password()
 - Dešifruje heslo ze Fernet formátu
5. GUI zobrazí heslo (nebo zkopíruje do schránky)
6. AuditManager zaznamenává přístup

Nastavení 2FA

1. Uživatel klikne "Nastavit 2FA" → TwoFASetupDialog
2. TwoFactorAuthManager.generate_secret_key() vytvoří sekreet
 - Generuje náhodných 32 bytů
 - Kóduje do Base32 formátu
3. Generuje QR kód (qrcode knihovna)
 - Formát: otpauth://totp/aplikace?secret=...
4. Uživatel naskenuje QR kódem do Google Authenticator
5. Aplikace generuje 10 backup kódů

- `CryptoManager.generate_backup_codes()`
- Uloží šifrované do `twofa.db`

6. `AuditManager` zaznamenává nastavení 2FA

Bezpečnostní architektura

Vrstvy bezpečnosti

Navržený systém je založen na vícevrstvé bezpečnostní architektuře, která kombinuje několik nezávislých mechanismů ochrany dat. Každá vrstva řeší jiný aspekt bezpečnosti a jejich kombinace zvyšuje celkovou odolnost systému vůči útokům.

První vrstva zajišťuje autentizaci uživatele. Přihlašovací heslo je zpracováno pomocí funkce PBKDF2 se SHA-256 a vysokým počtem iterací, čímž se zvyšuje odolnost vůči útokům hrubou silou. Pro porovnávání hodnot je využito časově bezpečné porovnávání, které minimalizuje riziko postranních kanálů (timing attacks).

Druhá vrstva představuje dvoufaktorovou autentizaci. Systém využívá algoritmus TOTP podle standardu RFC 6238, který generuje jednorázové kódy o délce šesti číslic s platností 30 sekund. Pro zvýšení dostupnosti jsou implementována také záložní jednorázová hesla. Systém zároveň brání opětovnému použití již použitých kódů (replay protection).

Třetí vrstva se zaměřuje na šifrování uložených dat. Hesla ke službám i auditní logy jsou šifrovány pomocí symetrického algoritmu (např. Fernet, využívající AES). Šifrovací klíč je odvozen z hlavního hesla uživatele, což zajišťuje, že data nelze dešifrovat bez znalosti tohoto hesla.

Čtvrtá vrstva zajišťuje integritu dat. Pro každou uloženou položku je generován autentizační kód pomocí HMAC-SHA256, který umožňuje detekovat jakoukoli neoprávněnou změnu databáze. V případě porušení integrity je tato skutečnost zaznamenána.

Pátá vrstva představuje auditní mechanismus. Systém zaznamenává všechny bezpečnostně relevantní události, jako jsou přihlášení nebo změny v databázi. Tyto záznamy jsou šifrovány a opatřeny časovou značkou, což umožňuje jejich pozdější analýzu a případnou forenzní kontrolu.

Ochrana hlavního hesla

Hlavní heslo uživatele není nikdy ukládáno v otevřené podobě. Při jeho zpracování je nejprve vygenerována náhodná sůl, která je následně použita spolu s heslem ve funkci PBKDF2-SHA256. Výsledkem je hash, který je uložen v databázi. Tento postup zajišťuje, že původní heslo nelze zpětně rekonstruovat a zároveň zvyšuje odolnost vůči útokům hrubou silou.

Ochrana hesel ke službám

Hesla jednotlivých služeb jsou před uložením šifrována. Šifrovací klíč je odvozen z hlavního hesla uživatele, což zajišťuje, že bez jeho znalosti nelze uložená data dešifrovat. Po zašifrování je k datům připojen autentizační kód (HMAC-SHA256), který slouží k ověření integrity. Tento mechanismus zaručuje, že hesla nejsou nikdy uložena v otevřené podobě a že jakákoli manipulace s daty je detekovatelná.

Architektura systému

Systém je navržen jako modulární aplikace složená z několika spolupracujících komponent. Uživatelské rozhraní, implementované pomocí knihovny PySide6, zajišťuje interakci mezi uživatelem a aplikací.

Modul pro správu uživatelů zajišťuje autentizaci a práci s hlavním heslem. Modul pro správu hesel implementuje ukládání, načítání a úpravu přihlašovacích údajů. Modul dvoufaktorové autentizace zajišťuje generování a ověřování jednorázových kódů.

Další části systému zahrnují modul integrity, který kontroluje neporušenost dat, a auditní modul, který zaznamenává bezpečnostní události. Kryptografické operace jsou centralizovány v samostatném modulu, který poskytuje funkce pro šifrování, dešifrování a odvozování klíčů.

Testovací metodika a vyhodnocení výsledků

Testovací metodika

Testování aplikace bylo zaměřeno na ověření funkčnosti jednotlivých částí systému a na kontrolu bezpečnostních mechanismů. Testy byly prováděny manuálně prostřednictvím grafického rozhraní aplikace a částečně také úpravou dat přímo v databázových souborech.

Funkční testování zahrnovalo základní scénáře práce uživatele s aplikací. Byla ověřena registrace nového uživatele, přihlášení pomocí správného hesla, zamítnutí přihlášení při zadání nesprávného hesla a správné fungování správy hesel. Dále bylo testováno ukládání nových hesel, jejich úprava, mazání a následné načítání.

Důležitou součástí testování bylo ověření dvoufaktorové autentizace. Bylo testováno přihlášení se správným TOTP kódem, odmítnutí přístupu při zadání nesprávného kódu a také použití záložních kódů pro obnovu přístupu.

Bezpečnostní testy byly zaměřeny na kontrolu ochrany uložených dat. Bylo ověřeno, že uživatelská hesla nejsou ukládána v otevřené podobě, ale pouze jako hash vytvořený pomocí PBKDF2-SHA256. U hesel ke službám bylo ověřeno, že jsou v databázi uložena v šifrované podobě a není možné je přečíst přímým otevřením databázového souboru.

Součástí testování byla také kontrola integrity dat. Test spočíval v ruční úpravě uloženého šifrovaného hesla v databázi a následném pokusu o jeho načtení v aplikaci. Aplikace v tomto případě správně detekovala změnu pomocí HMAC-SHA256 a zabránila zobrazení poškozených dat.

Dále byl testován auditní log, který zaznamenává všechny důležité operace v systému. Bylo ověřeno, že se zaznamenávají jak úspěšné, tak neúspěšné pokusy o přihlášení, a že uložené záznamy nejsou v databázi čitelné v otevřené podobě.

Vyhodnocení

Testování prokázalo, že aplikace splňuje všechny základní funkční požadavky. Registrace, přihlášení, správa hesel i dvoufaktorová autentizace fungují správně a podle očekávání.

Bezpečnostní testy potvrdily, že citlivá data nejsou ukládána v otevřené podobě a že je zajištěna jejich důvěrnost pomocí kryptografických metod. Mechanismus kontroly integrity úspěšně detekoval neoprávněné změny v databázi.

Mezi možné oblasti dalšího zlepšení patří zejména šifrování TOTP tajemství a bezpečnější ukládání záložních kódů. Tyto úpravy by dále zvýšily celkovou úroveň zabezpečení aplikace.

Celkově lze aplikaci hodnotit jako funkční a bezpečnou pro běžné použití, přičemž splňuje požadavky zadání a využívá moderní kryptografické přístupy.